SCS 2312 - Computational Models and Programming Language Concepts

Take Home Assignment

**Name:** D.Pirasannan
**Index No :** 23001461

## CFG.txt

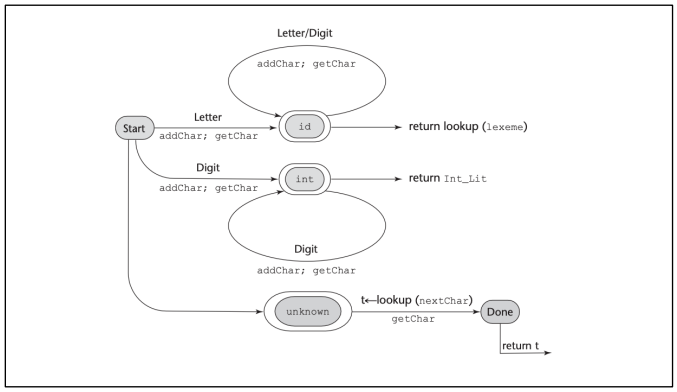
```
<program>         ::= <stmt_list>
<stmt_list>       ::= <stmt> | <stmt> <stmt_list>
<stmt>            ::= <declaration> | <print_stmt>
<declaration>     ::= int <identifier> = <expression> ;
<print_stmt>      ::= print ( <expression> ) ;

<expression>      ::= <term> | <term> + <expression> | <term> - <expression>
<term>            ::= <factor> | <factor> * <term> | <factor> / <term>
<factor>          ::= <identifier> | <number> | ( <expression> )

<identifier>      ::= [a-zA-Z]+
<number>          ::= [0-9]+
```

### 1. Lexical Analysis (The Tokeniser)

- tokenizer() function.

  The function reads the source file character-by-character to recognize lexemes. It uses a state-based approach:

  - Character is a letter, it transitions to an "Identifier/Keyword" state.
  - Character is a digit, it transitions to a "Integer Literal" state.
  - White spaces are ignored.




- A state diagram to recognize names, parentheses, and arithmetic operators -

### 2. Syntax Analysis (The Parser)

- parseProgram(), parseStatement(), parseExpression().

  The program uses Recursive Descent Parsing, a top-down parsing algorithm.

    - Each non-terminal in the CFG (like <statement> or <expression>) has a corresponding C function.
    - The parser uses a global currentToken variable as a "lookahead" to decide which grammar rule to apply next.
    - I have implemented the **operator precedence** as well, by spliting the logic into parseExpression, parseTerm, parseFactor to ensure multiplication happens before addition, as defined in the grammar.
- There is example Code block in the reference Book. **4.4**

### 3. Semantics & Symbol Table

- setVariable(), getVariable(), and the Variable struct.
    -
    - Static Semantics: The parser enforces rules that the CFG cannot, such as ensuring a variable is declared before use (getVariable check) and preventing re-declaration (isVariableDeclared check).
    - Dynamic Semantics (Execution): The values are stored in an array-based Symbol Table, representing the program's state (memory) during execution.

### 4. Error Handling

- expect() and getVariable().
    - Syntax errors are Detected when the currentToken does not match the expected terminal.
    - The expect() function enforces correct syntax structure.
    - Semantic Errors are Detected when logical rules are violated (e.g. Division by Zero in parseTerm, or Undefined Variables in getVariable).

Specially there are some addons like operator precedence and commented braces detection code. No relevant to the test code provided in the assignment.

---

Reference Book:

- Sebesta, R. W. Concepts of Programming Languages. Pearson.